Autonomous Smart Dual Infusion and Patient Monitoring System Using ESP32

Md. Turjo Hasnat, Md. Khalid Hasan, Moieb Al Jawad,
Swarup Deb Nath, and Md Shakib
Student IDs: 0112410067, 0112410009, 0112410289, 0112230150, 0112410546
[ADD DEPARTMENT], *[ADD UNIVERSITY]*
[ADD CITY, COUNTRY] — *[ADD AUTHOR EMAILS OR ORCIDs]*
Course: CSE 4326 — Microprocessor and Microcontroller Project
Supervisor: *[ADD SUPERVISOR NAME]*

***Abstract*—Manual supervision of intravenous fluid delivery and separate bedside monitoring can make it difficult to coordinate fluid status, patient observations, and caregiver intervention. This project addresses that integration problem through an educational embedded prototype called MediSmart. The implementation combines an ESP32 with a MAX30102 optical sensor, an AD8232 electrocardiogram front end, and an HX711 load-cell interface. Its firmware estimates pulse rate, oxygen saturation, and remaining fluid volume, drives saline, syringe, and oxygen-designated servo mechanisms, and controls a local buzzer. A Firebase Realtime Database connects device telemetry and control settings to doctor, patient, and family web interfaces. The supplied photographs, source code, and dashboard captures document the assembled platform and a consistent 324.1 mL displayed/database fluid estimate with an open-valve state above a 320 mL stopping threshold. Measurement accuracy, actual delivered dose, end-to-end latency, power efficiency, and clinical suitability remain unquantified because the supplied materials contain no calibrated experimental dataset.**

***Index Terms*—ESP32, intravenous monitoring, syringe actuation, MAX30102, AD8232, HX711, Firebase, embedded systems**

I. Introduction

A. Problem Statement and Motivation

Intravenous (IV) fluid delivery requires attention to the fluid source, the administration mechanism, and the patient's condition. In a manually supervised arrangement, a caregiver checks the container, adjusts the tubing or clamp, and separately observes vital signs. These activities motivate a common interface that reports fluid status and sensor availability while allowing supervised intervention. Research on remote infusion monitoring has explored optical drop detection and container-weight measurement to support this process [1].

The project presentation identifies manual IV and syringe operation, limited continuous monitoring, and repetitive caregiver tasks as the motivating problems. It also identifies affordability and suitability for resource-constrained hospitals in Bangladesh as design goals. These motivations describe the intended application; the provided materials do not contain a hospital deployment, stakeholder interview study, or measured reduction in caregiver workload.

MediSmart brings an IV stopping mechanism, a syringe mechanism, physiological sensing, and remote visualization onto a shared ESP32 platform. The engineering problem is to connect sensors with different interfaces, process their outputs, maintain actuator states, and exchange settings and telemetry over a network without blocking the local control sequence. The project is therefore relevant to microcontroller interfacing, digital signal processing, real-time task design, electromechanical actuation, and web-based monitoring.

B. Objectives and Scope

The implemented objectives are to estimate remaining fluid from a load-cell reading; acquire optical pulse and oxygen-saturation signals and an ECG analog signal; operate servo channels according to programmed conditions; activate a local buzzer during the firmware's low-oxygen condition; and expose monitoring/control views through Firebase. The term *dual infusion* refers here to the saline and syringe mechanisms described by the project, rather than to two independently measured and calibrated infusion channels. A third servo channel is designated for an oxygen-related mechanism.

The scope of this report is the archived prototype. Source code establishes the programmed behavior, photographs establish visible construction, and screenshots establish captured interface states. None of these alone establishes calibrated volume delivery or safe therapeutic operation. Measured accuracy, actual response time, and energy efficiency are therefore identified as evaluation tasks rather than assigned numerical values.

C. Literature Review

Lee, Yoon, and Kim [1] combined an optical sensor with a load cell for remote IV monitoring. Their system reports remaining fluid, infusion rate, and expected completion time to caregivers, and the study examines motion-related measurement limitations. This provides a useful comparison because MediSmart also uses container-weight information, but its archived implementation does not combine that reading with an optical drop counter or compute an infusion-rate estimate.

Contardi, Morikawa, Brunelli, and Thomaz [2] demonstrated a MAX30102 optical module connected to an ESP32 for remote heart-rate and oxygen-saturation monitoring. Their work supports the feasibility of integrating optical biosensing

with a networked microcontroller. MediSmart extends that integration into a prototype that also includes ECG acquisition, fluid estimation, and servo mechanisms; this difference does not demonstrate superior physiological measurement accuracy.

Kok and colleagues [3] developed a cloud-connected infusion platform with remote supervision through Blynk. Their design includes a stepper-driven pumping mechanism and infusion-related sensing, illustrating the importance of observing delivery as well as commanding a motor. MediSmart instead uses Firebase and three PWM servo channels in the archived firmware. Its saline threshold and syringe motion sequence are simpler than a calibrated infusion-rate controller.

Table I summarizes the features relevant to this project. The comparison concerns the cited implementations and the supplied MediSmart archive; it is not a performance benchmark.

D. Project Contributions and Distinguishing Features

The project's contribution is the integration of fluid estimation, physiological acquisition, local actuation, and several caregiver-facing views. The doctor interface changes the remaining-volume rules and displays device-reported valve and syringe states. The patient and family interfaces provide simpler pulse/oxygen displays, with an on-page family alert for programmed conditions. A separate network task exchanges settings while local firmware evaluates the actuator conditions.

These are distinguishing features of this prototype relative to the selected examples, rather than claims that no previous project has implemented them. The archived design also differs from the early presentation: load-cell sensing, AD8232 acquisition, and an oxygen-designated servo appear in the code and wiring; a stepper driver sequence, LCD driver, IR drop-counting routine, and ESP-NOW link do not. Subsequent chapters describe the implementation that can be traced to the supplied evidence.

II. Proposed Method

A. Block Diagram of the Overall System

Fig. 1 presents the implemented signal and control paths. Three sensing inputs feed the ESP32: the MAX30102 supplies red/infrared optical samples over I^2C, the AD8232 supplies an analog ECG signal and lead-off indications, and the HX711 supplies digitized load-cell data. The controller processes the samples and drives three servo channels and a buzzer. Wi-Fi connects a separate firmware task to Firebase, which stores current telemetry and remote settings.

B. Function of Each Subsystem

1) Controller and Local Scheduling: The ESP32 is configured as a Wi-Fi station. Its main loop reads sensors, calculates conditions, updates servo angles, and prints diagnostics. A FreeRTOS task performs Firebase HTTP requests separately. The main loop ends with a 20 ms delay, while the network task ends with a 2000 ms delay; actual periods also include computation, sensor waiting, request duration, and scheduling.

The network task obtains `/settings.json` with GET and updates `/vitals.json` with PATCH. This separates the slower network exchanges from the actuator sequence, but shared variables are not protected by a snapshot or mutex. The design is a single-device implementation: the database paths do not include a patient or device identifier.

2) Optical Pulse and Oxygen Estimation: The MAX30102 contains red/infrared optical sensing and a digital interface [4]. The firmware uses I^2C address `0x57` at 100 kHz and reads six FIFO bytes per loop to reconstruct the two 18-bit values. An infrared value below 20000 clears the reported pulse and oxygen values, representing no detected usable finger signal.

For a valid optical input, a running DC estimate for channel sample $x[n]$ is

$$D[n] = 0.95D[n-1] + 0.05x[n], \qquad A[n] = x[n] - D[n]. \tag{1}$$

The pulse routine detects an infrared AC threshold crossing followed by a decreasing sample. It imposes an interval greater than 400 ms between accepted beat events and calculates

$$\mathrm{BPM}_{\mathrm{candidate}} = \frac{60000}{\Delta t_{\mathrm{ms}}}. \tag{2}$$

Only candidates strictly between 50 and 130 BPM enter an eight-value buffer. Once at least five values exist, the routine sorts them, removes the smallest and largest, and averages the remainder. The displayed estimate applies additional smoothing with coefficients 0.97 and 0.03. These choices reduce visible variation but delay changes and exclude pulse rates outside the acceptance range.

The oxygen estimate uses peak-to-peak AC amplitudes and DC levels:

$$R = \frac{A_{\mathrm{red,pp}}/D_{\mathrm{red}}}{A_{\mathrm{IR,pp}}/D_{\mathrm{IR}}}, \qquad S_{\mathrm{candidate}} = 110 - 25R. \tag{3}$$

The candidate is restricted to 80–100%, then smoothed as $S[n] = 0.9S[n-1] + 0.1S_{\mathrm{candidate}}$ after initialization. These are the implemented algorithm and output limits, not verified oxygen-saturation accuracy. The archive supplies no reference-device comparison or calibration record.

3) ECG Acquisition: The AD8232 is a single-lead signal-conditioning front end designed to interface with an ADC [5]. Its output feeds ESP32 ADC1 channel 6 on GPIO 34; LO+ and LO− feed GPIO 25 and GPIO 26. If either lead-off input is high, the firmware sets the ECG value to zero. Otherwise it reads one raw ADC sample. The web chart receives only the latest cloud-published sample and therefore does not reproduce the complete locally acquired waveform.

4) Load-Cell Fluid Estimate: The HX711 serial interface is read through data and clock GPIOs. After the firmware's 24-bit conversion, a nonzero transformed count $W[n]$ is mapped to the nonnegative fluid proxy

$$V_{\mathrm{raw}}[n] = \max\left(0, \frac{W[n] - 8388608}{1000}\right). \tag{4}$$

When the stored estimate is zero, the new value is adopted directly; otherwise the update is

$$V[n] = 0.85V[n-1] + 0.15V_{\mathrm{raw}}[n]. \tag{5}$$

TABLE I
FEATURE COMPARISON WITH SELECTED PUBLISHED WORK

Work	Main sensing and mechanism	Communication and interface	Relation to MediSmart
Lee, Yoon, and Kim [1]	Optical drop detection and load-cell monitoring	Remote IV information and notifications	Provides a multisensor comparison; MediSmart's current fluid estimate uses HX711/load-cell input only.
Contardi and colleagues [2]	MAX30102 pulse and oxygen-saturation sensing with ESP32	Remote physiological visualization	Shares the optical sensing/controller combination; MediSmart also incorporates fluid and actuator functions.
Kok and colleagues [3]	Stepper-driven infusion and delivery-related sensors	Blynk web/mobile supervision	Provides an alternative cloud and pumping architecture; MediSmart implements servo motion and Firebase settings.
MediSmart archive	MAX30102, AD8232, HX711; saline, syringe, and oxygen-designated servos	Shared Firebase telemetry and doctor/patient/family views	Integrates the functions in one educational prototype; calibrated delivery and physiological accuracy are unverified.

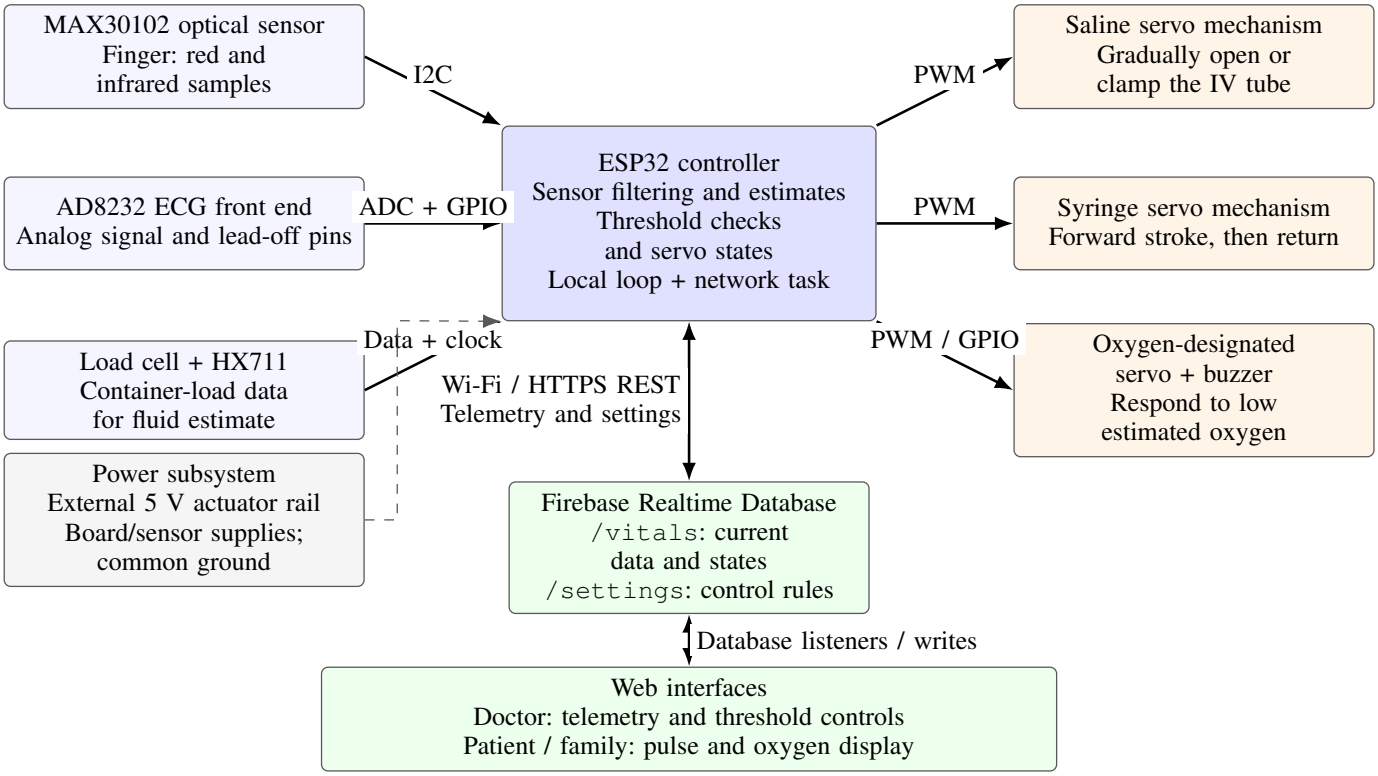


Fig. 1. Overall implementation architecture reconstructed from the firmware, website, and supplied wiring diagrams. Power connections are summarized; the separate wiring diagrams show the detailed rails. Servo arrows indicate commands, without measured flow or position feedback.

The firmware and dashboard express V in mL. A physical weight-to-volume measurement also requires a documented tare, calibration factor, mounting condition, and fluid-density assumption. Since these are absent, this report describes V as a remaining-fluid estimate or proxy rather than a calibrated volume measurement. A timeout retains the previous estimate without an explicit invalid-data flag.

C. Actuator Rules and Motion Sequences

Three LEDC channels share a 50 Hz, 13-bit PWM timer. The ideal angle-to-pulse relation for servo angle θ is

$$t_p(\theta) = 500 + \frac{2000\theta}{180} \quad (\mu\text{s}), \qquad 0 \le \theta \le 180. \tag{6}$$

The nominal frame duration is 20000 μs. The implementation uses integer division for the pulse-width term and then quantizes duty to the timer's resolution. Commanded angle is

the software state; no external position sensor establishes the mechanical angle.

1) Saline Stopping Rule: For configured threshold T_s, the clamp condition is

$$C_s = (T_s > 0) \wedge (V < T_s). \tag{7}$$

When true, the saline angle increases by 2° per loop toward 180° and the reported state progresses from *Closing* to *Stopped*. Otherwise the angle decreases by 2° toward zero and the state progresses from *Opening* to *Open*. Equality does not initiate closing. This is a remaining-volume stop rule; it does not command a continuous infusion rate in mL/h.

2) Syringe Sequence: For threshold T_d, the volume-based condition is $(T_d > 0) \wedge (V < T_d)$. The firmware also checks whether the current local hour and minute match `dose_time`, if such a setting has been parsed. Either condition starts a cycle when no cycle is active. The servo advances by 1° per loop toward 180°, reports *Pushing*, then requests a cloud reset of `dose_trigger_ml` and returns by 2° per loop toward zero. The final state is *Standby*.

No programmed syringe volume, plunger-position measurement, or flow calibration is supplied. The web reset button clears a trigger value and does not implement an immediate abort of an active cycle. The time condition has no once-per-event latch, so additional cycles can start while the matching minute remains active. These limitations are considered in Section V.

3) Oxygen-Designated Servo and Buzzer: The local condition is $0 < S < 85\%$. When true, the firmware sets the buzzer GPIO high and advances the oxygen-designated servo by 2° per loop. Otherwise it silences the buzzer and returns the servo toward zero. The 85% value is a project constant and is not presented as a clinically approved treatment threshold. The photographs include a mask and tubing, but no oxygen-flow sensor or measured delivery result establishes the mechanism's therapeutic effect.

D. Cloud Data and Web Interfaces

Table II lists the active cloud fields. The doctor page listens for telemetry and settings changes, plots a rolling buffer of 50 received ECG values, and writes two fluid thresholds. Its open button sets $T_s = 0$; its close button sets $T_s = 9999$, making the clamp condition true when the estimated volume is below that sentinel. The reset button sets $T_d = 0$.

Registration uses Firebase email/password authentication and stores a selected patient, family, or doctor role. Login reads this role to select a page. Patient and family pages include a signed-in check; the archived doctor page lacks an authentication guard and retains configuration placeholders. Therefore the report distinguishes role-based page routing from verified role-based authorization, which requires server-enforced access rules [6].

The patient view labels a zero reading as *Sensor Ready / Place Finger*. The family view displays an alert for BPM above 120, positive BPM below 50, or positive SpO_2 below 92%. The doctor view uses positive SpO_2 below 85% for its oxygen warning and interprets an ECG value of zero as lead disconnection. These inconsistent prototype display thresholds are documented rather than reconciled by assuming a common medical policy.

TABLE II
IMPLEMENTED FIREBASE DATA CONTRACT

Path	Fields	Purpose
`/vitals`	`bpm`, `spo2`, `ecg`, `current_ml`	Latest processed estimates and raw ECG sample.
`/vitals`	`saline_status`, `dose_status`	Software-reported servo sequence states.
`/settings`	`saline_stop_ml`, `dose_trigger_ml`	Remaining-volume control thresholds.
`/settings`	`dose_time`	Firmware-only hour/minute trigger; no corresponding web control.
`/users/<uid>`	`email`, `role`	Registration record used for login routing.

III. IMPLEMENTED HARDWARE SYSTEM

A. Hardware Equipment and Model Identification

Table III records the hardware supported by the firmware, wiring sketches, and photographs. Exact model information is retained where documented, and ambiguous identities are marked for confirmation. The photographed load cell has a readable 5 kg label, while the ESP32 development-board variant and power-source rating are not established by readable labels.

The presentation additionally lists an IR drop sensor, stepper motor, lead screw, LCD, rechargeable battery, and following sensor/camera. The supplied medical firmware contains no corresponding drop-counting, stepper, LCD, or following-control routines. These items are not presented as verified functioning modules in the implementation inventory.

B. Electrical Interfacing and Pin Assignment

Table IV gives the code/wiring pin assignment. The MAX30102 communicates through SDA and SCL; the AD8232 provides an ADC input and two digital status inputs; and the HX711 uses a clock/data interface. The three servo signals use dedicated LEDC channels, while the buzzer is switched through a GPIO output.

The supplied sensor sketch shows a 3.3 V rail and a common ground. The actuator sketch shows an external 5 V rail with servo and ESP32 grounds joined; it does not establish the supply's available current or transient performance. The diagrams also omit detailed load-cell bridge wiring, protective components, and module-specific regulator arrangements. Module voltages, I^2C electrical behavior, and power capacity therefore require verification against the actual boards and device documentation [4], [5], [7].

C. Software Environment

The embedded program is C, with ESP-IDF as the framework and PlatformIO as the project environment. The configuration specifies the `espressif32` platform, `esp32dev`

TABLE III
HARDWARE INVENTORY AND EVIDENCE OF COMPONENT IDENTITY

Equipment	Model / specification	Role and evidence
Microcontroller development board	ESP32; PlatformIO target `esp32dev`	Main acquisition, actuation, Wi-Fi and task scheduling. Exact board/module variant requires confirmation.
Optical biosensor module	MAX30102	Red/infrared acquisition for pulse and SpO_2 estimates; named in code, slides, and sensor wiring.
Single-lead ECG front end	AD8232	Analog signal conditioning and lead-off outputs; named in the sensor wiring, with matching firmware pins.
Load-cell ADC/interface	HX711	Reads the container-load sensor; identified by firmware and wiring.
Load cell and hook	5 kg capacity label; exact part number unspecified	Upper stand assembly visible in photograph 4. Mechanical mounting converts supported load to a sensing input.
Saline clamp servo	SG90 in wiring; MG996R in code comments	PWM output on GPIO 5. Physical model must be confirmed because the two sources disagree.
Syringe and oxygen-designated servos	MG996R in wiring and code comments	PWM outputs on GPIO 18 and GPIO 16. Actual torque/travel and delivered quantities were not measured in supplied records.
Local alarm	Active buzzer; exact part number unspecified	GPIO 4 output, named in the actuator diagram and firmware.
Syringe mechanism	Syringe, handmade guide, rack and gear	Photographed mechanism; syringe capacity and travel-to-volume calibration are unspecified.
Fluid/oxygen-related assembly	Tubing, mask, and mounted mechanical components	Visible in photographs 2, 3, and 6; exact regulator/container ratings and output flow are unspecified.
Structure and power	Pole, caster base, enclosure, breadboards, wiring; external 5 V actuator rail in diagram	Provides physical integration. Battery chemistry, capacity, regulator identity, and current budget require confirmation.
Additional photographed hardware	Motor-driver-like board, secondary controller, ultrasonic-style modules	Visible in photographs but not operated by the supplied ESP32 source. Exact models and any separate mobility firmware are unspecified.

TABLE IV
ESP32 PIN ASSIGNMENT

Signal	GPIO	Interface
MAX30102 SDA / SCL	21 / 22	I^2C, 100 kHz
AD8232 OUT	34	ADC1, channel 6
AD8232 LO+ / LO−	25 / 26	Digital inputs
HX711 DT / SCK	19 / 23	Serial data / clock
Saline servo	5	LEDC channel 1
Syringe servo	18	LEDC channel 2
Oxygen-designated servo	16	LEDC channel 0
Active buzzer	4	Digital output

board, DIO flash mode, and a 115200-baud serial monitor. CMake requires version 3.16 or later and registers the sources under `src`. The saved generated SDK configuration identifies ESP-IDF 6.1.0, but the PlatformIO platform version is not pinned, so a fresh installation is not guaranteed to reproduce that toolchain exactly.

The firmware uses ESP-IDF GPIO, LEDC, I^2C, ADC, Wi-Fi, HTTP-client, SNTP, and FreeRTOS facilities. The frontend consists of static HTML, CSS, and JavaScript pages served by Firebase Hosting. Application pages use Firebase SDK 8.10.1; the doctor interface also uses Bootstrap 5.3.0 and a Chart.js CDN dependency. The default hosting welcome page uses a different Firebase SDK version and is separate from the application login page.

A configuration inconsistency affects reproducibility: `platformio.ini` specifies a 4 MB upload flash size, whereas the saved SDK configuration selects 2 MB. The actual board capacity must be checked and both settings made consistent before reproducing a firmware build. There are no custom tests in the supplied test directory, and no firmware build or hardware experiment is claimed as part of this report-generation process.

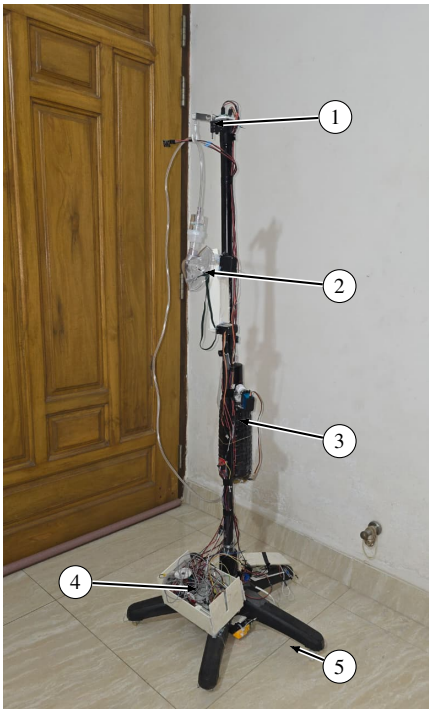
D. Figures of the Implemented Hardware System

Fig. 2 labels the overall structure and two important mechanical assemblies. The upper load-cell/hook supports load sensing, the tubing and mask identify the respiratory-related assembly, and the lower enclosure contains the wiring and controller area. The syringe close-up shows a rack-and-gear construction rather than a verified lead-screw drive. Fig. 3 identifies the base electronics and electrode connections.

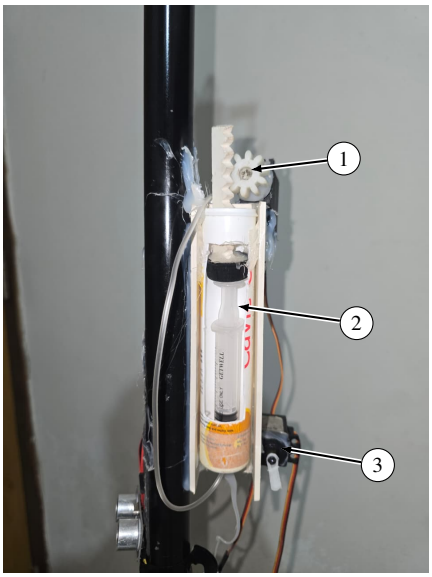
IV. RESULTS

A. Available Evidence and Evaluation Method

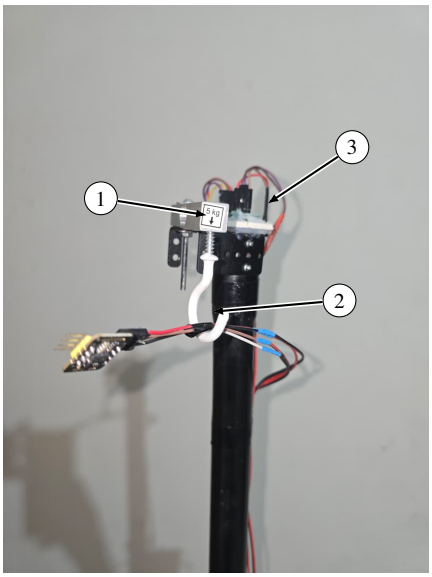
The available evidence comprises the assembled-system photographs, the implementation source, and saved website/database screenshots. The slides provide objectives and feature descriptions, but no tabulated trials, reference measurements, instrument readings, timestamps, or experimental logs. The results below therefore report directly visible states and source-derived behavior separately. Numerical accuracy, dose error, and actual latency cannot be calculated from the archive.



(a) Overall prototype. 1: load-cell/hook; 2: mask/tubing; 3: mounted actuation assembly; 4: electronics enclosure; 5: caster base.



(b) Syringe construction. 1: rack and gear; 2: syringe/guide; 3: visible servo attachment. Photographs do not quantify delivered dose.



(c) Load sensing mount. 1: load cell labeled 5 kg; 2: hanging hook; 3: wiring and support electronics.

Fig. 2. Annotated photographs supplied with the project: overall view (photograph 6), syringe assembly (photograph 3), and load-cell mount (photograph 4). Numbered callouts identify visible parts without assigning unreadable model labels.

B. Captured Dashboard and Database State

Fig. 4 shows the doctor interface with a remaining-fluid estimate of 324.1 mL, a saline stop setting of 320 mL, an *Open* valve state, and a *Standby* syringe state. The supplied Firebase console capture shows the same `current_ml`, threshold, and status values. This agreement supports consistency between the captured dashboard and stored data at that observation; it does not establish the correctness of the load-cell calibration or prove a complete actuation sequence.

The state also agrees with the code's strict remaining-volume rule. Since $324.1 < 320$ is false, (7) does not request closure. This interpretation is important because the threshold is a remaining-volume cutoff, rather than a target volume already delivered. Screenshots 6 and 7 duplicate screenshots 1 and 2 respectively and are not counted as additional trials.

Table V records the observation without assigning unmeasured performance values. The additional `saline_ml` and `saline_threshold` entries visible in the console screenshot are not read or written by the current active dashboard/firmware paths, and are not treated as the implemented data contract.

TABLE V
VALUES VISIBLE IN THE SUPPLIED CAPTURES

Quantity	Visible value	Supported interpretation
Fluid estimate	324.1 mL	Displayed estimate; calibration unverified.
Saline stop setting	320 mL	Positive remaining-volume cutoff.
Saline state	Open	Consistent with estimate above cutoff.
Dose trigger	0	Volume-trigger rule disabled in console snapshot.
Syringe state	Standby	No stroke shown at capture time.
Pulse / oxygen	0 / 0	Unavailable readings; no physiological accuracy evidence.
ECG	0	Doctor view shows disconnection warning and zero plot.

C. Patient and Family Interface Outputs

Fig. 5 shows the patient dashboard labeling the zero pulse and oxygen values as *Sensor Ready / Place Finger*. Fig. 6 shows the corresponding family view. The family implementation excludes zero values from its positive-low-reading tests, so the lack of a family alarm in this capture is consistent with its code. It does not demonstrate that an abnormal valid measurement was detected or that a notification reached a caregiver.

The account-creation screenshot contains the patient, family/guardian, and medical-doctor role choices. Source inspection confirms the intended authentication and routing sequence. Successful registration of every role, enforcement of doctor privileges, and protected access to telemetry are not

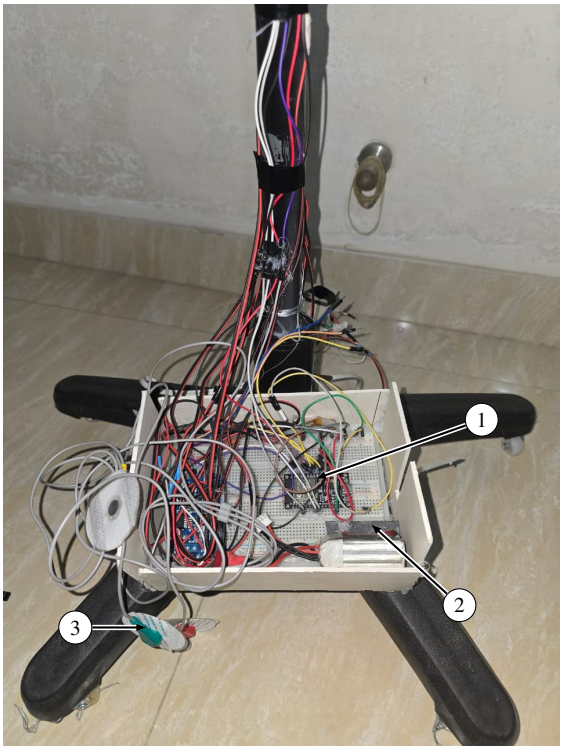


Fig. 3. Base electronics in photograph 1. 1: controller-board area; 2: breadboard wiring; 3: electrode pads/cables. Wiring obscures some labels, so exact component identities are taken from the source/wiring inventory rather than inferred from this view.

established by that single screenshot.

D. Source-Derived Output Conditions

The prototype's key output conditions are explicit in the firmware. The following excerpt is taken from the control-condition portion of `main.c`; it contains no credentials:

```
bool saline_stop_active =
  (saline_stop_ml > 0 &&
   current_ml < saline_stop_ml);
bool weight_trigger =
  (dose_trigger_ml > 0 &&
   current_ml < dose_trigger_ml);
bool o2_emergency =
  (smooth_spo2 > 0 &&
   smooth_spo2 < OXYGEN_DANGER_LEVEL);
```

When `o2_emergency` is true, the output branch calls `gpio_set_level(BUZZER_PIN, 1)` and advances the oxygen-designated servo angle. The code therefore establishes the command path from an estimated sensor value to a GPIO/PWM output. The supplied photographs do not capture the buzzer energized or provide measured oxygen flow, so physical alarm activation and treatment delivery are not reported as observed results.

Table VI presents illustrative boundary cases derived from these conditions. These are analytic examples, not newly conducted bench tests or recorded patient data.

E. Timing and Efficiency That Can Be Inferred

The 20 ms programmed loop delay and angular increments yield nominal delay-only travel times: 90×20 ms $= 1.8$ s for a 180° saline/oxygen command, 180×20 ms $= 3.6$ s for the syringe's forward stroke, and about 1.8 s for its return. Sensor access and transition iterations add time, and mechanical motion may differ from the command sequence. These values are calculations from the implementation, not measured response times.

The cloud task delays two seconds after completing its requests, so it offers periodic latest-value publication with additional HTTP overhead. The doctor plot appends one ECG value per database callback rather than a timestamped waveform packet. Consequently, the captured flat plot and software timing do not establish ECG waveform fidelity. No current measurements, battery-capacity specification, deployment uptime, or itemized purchase costs are provided, so energy efficiency, battery life, and total cost remain unquantified.

F. Experimental Validation Required for Performance Claims

Table VII specifies the missing measurements and an appropriate way to obtain them. These procedures are proposed future evaluation, not completed experiments. Fluid and syringe trials should use a laboratory collection setup; any human physiological study requires the relevant supervision and consent arrangements.

For N paired reference and estimated volumes, a reproducible accuracy summary would include

$$\mathrm{MAE}_V = \frac{1}{N}\sum_{i=1}^{N} |\hat{V}_i - V_i^{\mathrm{ref}}| \quad (\mathrm{mL}). \tag{8}$$

An end-to-end control measurement would separately record database write time, device receipt time, and physical actuator response. No values for these metrics are inserted because the required paired records and timestamps are absent.

V. Discussion

A. Limitations and Examples of Unexpected Operation

1) Calibration and Unobserved Delivery: The current fluid conversion has a fixed offset/divisor and no supplied calibration, tare, or density record. Moving the stand, changing the supported container, or changing mechanical loading can therefore produce an unreliable remaining-volume estimate. The syringe and oxygen-designated mechanisms receive angle commands without delivered-volume, pressure, flow, or external position feedback. A displayed *Stopped* or *Pushed* state represents the firmware's sequence state and may not match physical delivery if a linkage slips or a servo stalls.

2) Repeated Dose Scheduling and Reset Semantics: A configured time trigger remains true throughout its matching minute. Once the syringe completes its forward/return cycle, the same condition can start another cycle; no executed-event identifier or date latch prevents repetition. Clearing the browser's dose threshold also does not abort an active cycle or clear a previously parsed time schedule. These are concrete examples in which user expectations and the programmed behavior may differ, and require an explicit once-per-event schedule and cancel/acknowledgment mechanism before a dosing claim is justified.

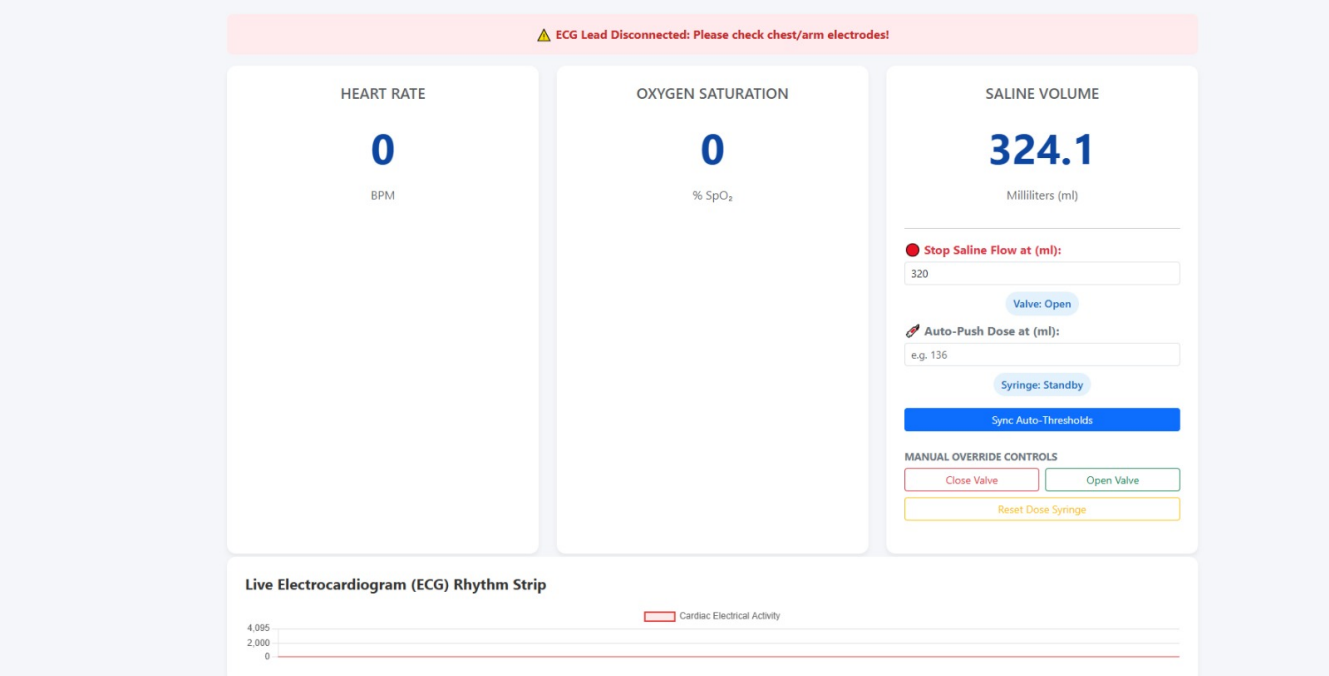


Fig. 4. Supplied doctor dashboard capture: estimated fluid 324.1 mL, configured stop threshold 320 mL, valve Open, syringe Standby, and unavailable physiological values. The zero ECG trace and disconnection banner document an interface state, without establishing a valid measured rhythm.

TABLE VI
ILLUSTRATIVE INPUT-TO-OUTPUT CASES DERIVED FROM THE FIRMWARE

Case	Assumed inputs	Programmed outcome	Evidence status
Captured saline state	$V = 324.1$ mL, $T_s = 320$ mL	Closing condition false; target moves toward open.	Values/status visible in saved captures and consistent with code.
Saline threshold equality	$V = 320$ mL, $T_s = 320$ mL	Strict comparison false; does not begin closing.	Boundary example from source only.
Below saline cutoff	$V = 319$ mL, $T_s = 320$ mL	Servo moves toward clamp; Closing then Stopped if condition persists.	Source-derived case; no matching observed sequence supplied.
Volume-triggered syringe	$V = 135$ mL, $T_d = 136$ mL; cycle inactive	Begins fixed forward/return angle sequence.	Source-derived case; delivered volume unmeasured.
Low estimated oxygen	$S = 84\%$ with usable optical input	Buzzer GPIO high; oxygen-designated servo advances.	Source-derived case; alarm/flow not experimentally documented.
Unavailable optical input	Infrared count below 20000	Pulse and oxygen estimates zero; positive-low-oxygen condition false.	Source branch; zero outputs are visible, but raw optical input was not recorded.
ECG lead off	LO+ or LO− high	ECG value forced to zero; doctor page shows warning.	Source branch; warning visible, input-pin state unrecorded.

3) Signal Availability and Alarm Meaning: The pulse buffer accepts only values strictly between 50 and 130 BPM; a signal outside that range can be rejected while the display retains prior buffered values. SpO_2 is restricted to 80–100%, so the interface cannot display lower estimates. Finger-signal loss sets the oxygen estimate to zero and disables the positive-low-oxygen alarm condition, making sensor unavailability distinct from an emergency reading. These behaviors need explicit signal-validity/freshness reporting rather than interpretation as a complete physiological alarm system.

The family view's low-pulse alarm requires a positive BPM below 50, while the firmware normally admits pulse estimates above 50. That combination limits the usefulness of the family low-pulse branch for this telemetry source. The patient view labels both low and high nonzero out-of-range pulse readings as *Elevated Heart Rate*, and the doctor page equates ECG zero with disconnected leads. The interfaces need consistent thresholds and separate validity fields to avoid misleading labels.

TABLE VII
PROPOSED VALIDATION PLAN; NO RESULTS ARE CLAIMED FOR THESE TESTS

Subsystem	Proposed procedure	Record and evaluation metric
Fluid estimate	Tare the empty assembly; apply known loads/volumes across its range; repeat static and disturbed trials.	Reference/estimated volume pairs; mean absolute error, maximum error, repeatability, and mounting conditions.
Saline stopping	Move the measured estimate through positive thresholds, equality, and disabled settings; collect remaining leakage after closure.	Input, GPIO/PWM time, physical closure time, state sequence, and residual fluid movement.
Syringe mechanism	Calibrate syringe geometry/travel using collected fluid; repeat full-stroke trials under specified loads.	Reference delivered volume, variation, backlash, stalls, and mechanical limits.
Pulse / SpO_2	Compare synchronized valid outputs with an appropriate reference instrument and document signal/motion conditions.	Paired readings, acquisition availability, agreement errors, and recovery after signal loss.
ECG channel	Use a known simulator/reference signal and record locally timestamped samples; exercise both lead-off inputs.	Sampling interval/jitter, signal agreement, lead-off detection, and cloud information loss.
Cloud/control	Timestamp a setting change, device receipt, output command, and browser status; repeat with delays/disconnection.	Median/maximum end-to-end times, failed writes, stale-data behavior, and recovery.
Power and operation	Measure voltage/current with Wi-Fi and servos active, and observe resets/stalls on the actual supply.	Peak current, energy per interval, operating duration, and brownout behavior.

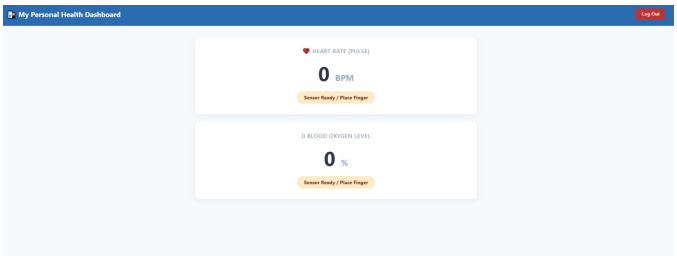


Fig. 5. Patient dashboard capture displaying zero readings and sensor-ready messages. The image demonstrates the unavailable-reading presentation rather than validated physiological sensing.

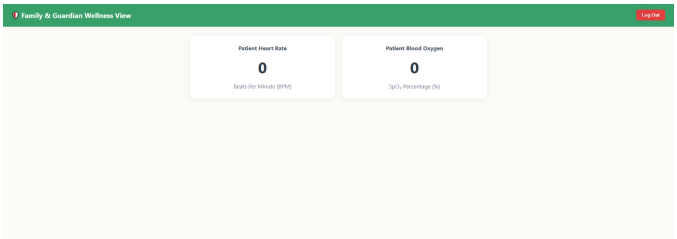


Fig. 6. Family/guardian dashboard capture. No alert is visible for the captured zero values, which the implemented alert condition excludes.

4) Network, Security, and Reproducibility: Telemetry has no timestamp, sequence number, stale-data indication, or device identifier. During a connectivity failure, the browser may retain old values and the firmware may continue with previous settings. The HTTP code performs limited status/error validation and uses text searches instead of a structured JSON parser; network and control tasks also share state without an atomic snapshot. These limitations affect command reliability and interpretation of the displayed values.

The archived configuration explicitly disables TLS server verification, and credentials are embedded in the source. The doctor page lacks an authentication guard, registration permits self-selection of the doctor role, and no database-rule file establishes the deployed authorization policy. Server-enforced access rules, authenticated device requests, certificate verification, command validation, and restricted telemetry access are required improvements [6], [8]. The shared database paths and flash-size/toolchain inconsistencies further limit reproducibility and extension to multiple patients.

B. Depth of Knowledge (P1)

The project requires knowledge across microcontroller architecture, ADC and serial interfaces, PWM timing, filtering, FreeRTOS tasks, network protocols, cloud data, and electromechanical design. It also requires understanding why a sensor output, a filtered estimate, a software actuator state, and an independently measured delivered quantity are different forms of evidence. These interacting domains support the depth-of-knowledge mapping for P1.

C. Conflicting Requirements and Trade-offs (P2)

Affordability and a single-controller design favor inexpensive sensors and servo mechanisms, while accurate delivery and robust monitoring require calibration, feedback, and additional fault detection. Strong smoothing improves display stability but delays changes; separating cloud requests from the local loop improves continuity but introduces shared-state synchronization requirements. The implementation therefore trades hardware simplicity and development effort against measurement fidelity, response predictability, and verified control.

D. Two Alternative Designs and Analysis (P3)

1) Alternative 1: Instrumented Stepper/Lead-Screw Syringe: A stepper motor, lead screw, homing switches, and

plunger-position/force sensing would make commanded displacement more explicit than a fixed servo sweep. With a measured syringe bore area A and plunger displacement Δx, the nominal geometric delivery is $\Delta V = A\Delta x$, but backlash, fluid compliance, and pressure still require collection-based calibration. This alternative adds motor-driver, mechanical, and sensing cost, yet offers a clearer route to volume/rate control and fault detection than the archived mechanism.

2) Alternative 2: Passive Monitor with Manual Infusion Control: A monitoring-only device could combine load-cell and optical drop sensing, retain the physiological channels, and alert a caregiver without remotely actuating fluid delivery. This would reduce actuator complexity and associated command hazards, while still requiring reliable calibration and alarm evaluation. It would provide less automation and retain manual response time, but could be a more focused first stage for validating sensing and caregiver usability. The optical/load-cell integration in prior work [1] provides a relevant design precedent.

E. Impact and Familiarity of Issues (P4)

The intended application extends beyond routine computer-science coursework into medical workflow, fluid mechanics, physiological measurement, and human interaction. The potential benefit is a common view of fluid estimates and sensor status that can support caregiver attention, particularly where equipment cost matters. That benefit remains an engineering objective; there is no supplied evidence of improved clinical outcomes or reduced staff workload, and those questions require domain expertise and evaluation with intended users.

F. Standards, Manuals, and Ethical Considerations (P5)

Relevant references include device documentation, the NXP I^2C specification, and ESP-IDF/Firebase manuals, although the archive contains neither electrical/timing compliance measurements nor a record of the manuals originally studied by the team [4]–[8]. The 100 kHz I^2C setting matches Standard-mode, while disabled TLS verification departs from authenticated server verification; use of a named protocol alone therefore does not establish complete compliance. The ACM code motivates honest performance claims, privacy, and avoidance of harm [9], while FDA infusion guidance provides a comparator for hazard analysis, alarm behavior, and evaluation [10], without establishing certification of this prototype or compliance with local clinical requirements.

G. Stakeholders and Requirement Incorporation (P6)

The implemented views identify doctors, patients, and family/guardians as intended stakeholders, and the slides also identify nursing workload as a motivation. Their represented requirements differ: clinicians need telemetry and controllable thresholds, patients need a simple display, and families need accessible status and warnings. However, no interview notes, stakeholder approvals, funding discussions, or usability-study results were supplied. P6 is therefore left unmarked until actual consultation and resulting design changes are documented; intended users alone do not demonstrate stakeholder involvement.

TABLE VIII
COMPLEX ENGINEERING PROBLEM MAPPING

P1	P2	P3	P4	P5	P6	P7
✓	✓	✓	✓	✓	—	✓

P1: depth of knowledge; P2: conflicting requirements; P3: depth of analysis; P4: familiarity of issues; P5: extent of applicable codes; P6: extent of stakeholders; P7: interdependence.

H. Module Interdependence (P7)

Sensor interfaces, local control, cloud synchronization, and browser views form recognizable modules, and the local loop can continue while the network task is delayed. Nevertheless, remote controls depend on Wi-Fi/Firebase and a shared settings schema, while browser status depends on device publication and both tasks share mutable variables. The physical sensors/actuators are coupled to firmware pins and calibration, so modular files and separate tasks provide partial separation rather than complete independence.

I. Complex Engineering Problem Mapping

Table VIII marks the aspects discussed with evidence or design analysis. P5 represents the documented interface/security/ethical considerations, not a certification claim. P6 remains pending because consultation evidence is absent.

VI. CONCLUSION

MediSmart integrates an ESP32, optical and ECG sensing, a load-cell fluid estimate, three servo-control channels, and a local buzzer with Firebase-based doctor, patient, and family interfaces. The archived implementation provides remaining-volume threshold control and a syringe motion sequence, with a captured 324.1 mL dashboard estimate and an open-valve state consistent with the configured 320 mL cutoff.

Separate local and network tasks support continued control processing during HTTP requests, but measured latency, accuracy, dose delivery, cost, and power efficiency are not established by the supplied evidence. Future development should prioritize fluid and syringe calibration, independently observed actuator/flow states, once-per-event scheduling, and explicit signal-validity reporting. Timestamped waveform acquisition, consistent alert policies, authenticated access, verified TLS, and device-specific data paths would strengthen monitoring reliability and extend the architecture. Controlled bench evaluation and documented stakeholder feedback should precede any claim of clinical benefit; the project repository and demonstration links are reserved in the references for later completion [11], [12].

ACKNOWLEDGMENT

[ADD verified acknowledgments, including supervisor, laboratory support, borrowed equipment, or funding if applicable.]

REFERENCES

[1] J.-K. Lee, K.-C. Yoon, and K. G. Kim, "Design of a remote monitoring system based on optical sensors to prevent medical accidents during fluid treatment," *Applied Sciences*, vol. 11, no. 21, Art. 10124, 2021, doi: 10.3390/app112110124. [Online]. Available: https://www.mdpi.com/2076-3417/11/21/10124.

[2] U. A. Contardi, M. Morikawa, B. Brunelli, and D. V. Thomaz, "MAX30102 photometric biosensor coupled to ESP32-webserver capabilities for continuous point of care oxygen saturation and heartrate monitoring," *Engineering Proceedings*, vol. 16, no. 1, Art. 9, 2022, doi: 10.3390/IECB2022-11114. [Online]. Available: https://www.mdpi.com/2673-4591/16/1/9.

[3] C. L. Kok, C. K. Ho, T. K. Lee, Z. Y. Loo, Y. Y. Koh, and J. P. Chai, "A novel and low-cost cloud-enabled IoT integration for sustainable remote intravenous therapy management," *Electronics*, vol. 13, no. 10, Art. 1801, 2024, doi: 10.3390/electronics13101801. [Online]. Available: https://www.mdpi.com/2079-9292/13/10/1801.

[4] Analog Devices / Maxim Integrated, "MAX30102: High-sensitivity pulse oximeter and heart-rate sensor for wearable health," data sheet, Rev. 1, Oct. 2018. [Online]. Available: https://www.analog.com/media/en/technical-documentation/data-sheets/max30102.pdf. Accessed: Oct. 8, 2026.

[5] Analog Devices, "AD8232: Single-lead, heart rate monitor front end," data sheet, Rev. D. [Online]. Available: https://www.analog.com/media/en/technical-documentation/data-sheets/ad8232.pdf. Accessed: Oct. 8, 2026.

[6] Google Firebase, "Understand Firebase Realtime Database Security Rules." [Online]. Available: https://firebase.google.com/docs/database/security. Accessed: Oct. 8, 2026.

[7] NXP Semiconductors, "I^{2}C-bus specification and user manual," UM10204, Rev. 7.0, Oct. 1, 2021. [Online]. Available: https://www.nxp.com/docs/en/user-guide/UM10204.pdf.

[8] Espressif Systems, "ESP HTTP Client," *ESP-IDF Programming Guide*. [Online]. Available: https://docs.espressif.com/projects/esp-idf/en/stable/esp32/api-reference/protocols/esp_http_client.html. Accessed: Oct. 8, 2026.

[9] Association for Computing Machinery, "ACM Code of Ethics and Professional Conduct," Jun. 22, 2018. [Online]. Available: https://www.acm.org/code-of-ethics.

[10] U.S. Food and Drug Administration, "Infusion pumps total product life cycle: Guidance for industry and FDA staff," Dec. 2, 2014. [Online]. Available: https://www.fda.gov/media/78369/download.

[11] MediSmart project team, "Project source-code repository." *[ADD PROJECT GITHUB REPOSITORY URL].*

[12] MediSmart project team, "Project demonstration video." *[ADD PROJECT YOUTUBE DEMONSTRATION URL].*

APPENDIX A
SETUP AND LABORATORY OPERATING INSTRUCTIONS

These instructions support supervised, nonclinical bench demonstrations. Use water and mechanical mock loads; keep the infusion, syringe, and oxygen-designated mechanisms disconnected from patients. Verify wiring, mechanical clearance, supply capacity, and a means of removing actuator power before starting. The supplied photographs and wiring sketches do not establish clinical or electrical safety.

A. Firmware Preparation

Install VS Code with PlatformIO IDE and the board's USB serial driver, then open `medismart/medismartVscode/` as the PlatformIO project. Select environment `esp32dev`, board `esp32dev`, and framework `espidf`; the application is ESP-IDF C. Regenerate the included editor/debug configuration because its absolute paths belong to another workstation. Record the installed toolchain and framework versions when reproducing the build.

In `src/main.c`, replace the Wi-Fi settings with *[LAB SSID]* and *[LAB PASSWORD]*, and configure matching settings/vitals endpoints for *[LAB FIREBASE DATABASE URL]*. Keep credential values out of submitted listings. Confirm the physical flash capacity: `platformio.ini` requests 4 MB, whereas `sdkconfig.esp32dev` selects 2 MB. Reconcile the saved configuration and board settings before uploading. The following commands run from the firmware project directory:

```
pio run -e esp32dev
pio run -e esp32dev -t upload
pio device monitor -b 115200
```

Select the correct serial port if automatic detection fails. Inspect build/upload output and the serial dashboard; a successful compilation alone does not verify connected sensors or mechanisms.

B. Cloud and Browser Configuration

Use an isolated laboratory database. Configure Firebase Realtime Database and email/password Authentication, then make the browser pages and firmware reference the same project. Replace the doctor page's API-key, sender-ID, and application-ID placeholders with that project's web configuration. The supplied `firebase.json` specifies static content in `medismart/public/`; it supplies no database rules.

Before protected operation, implement server-side rules that restrict telemetry writes to the device and settings changes to approved operators. Registration-selected roles must not authorize themselves. This requires corresponding firmware REST authentication and authentication/role checks in the doctor page, which are absent from the supplied code. Restore TLS server verification before protected deployment. Official rule and REST-authentication guidance is available at https://firebase.google.com/docs/database/security and https://firebase.google.com/docs/database/rest/auth; secure operation cannot be achieved by changing browser navigation alone.

Open `login.html` explicitly: the supplied root page remains Firebase's welcome template. Register a laboratory account using the patient, family, or doctor selection, then sign in. Login reads the saved role and redirects accordingly. Patient/family pages display the shared vital stream; the doctor page adds fluid estimates, statuses, settings, and a rolling received-value ECG plot. These screens provide navigation rather than verified role-based authorization in the current implementation.

C. Startup and Controls

After power-up, verify Wi-Fi, database updates, sensor contact, and known-weight/volume calibration before interpreting readings. SNTP uses `pool.ntp.org`; the POSIX timezone `UTC-6` represents Dhaka's UTC+06:00. Confirm actual synchronization separately because startup does not wait for valid time. Record local/remote observation times to identify stale telemetry.

Sync Auto-Thresholds writes both remaining-volume thresholds. A positive saline threshold closes its valve only when the

estimate is strictly lower; equality does not trigger closure. A positive dose threshold starts a fixed syringe stroke under the same comparison and does not specify delivered drug volume. Blank or invalid inputs become zero, disabling the corresponding volume condition.

Open Valve writes a zero saline threshold. *Close Valve* writes 9999, producing a closure condition only while the estimate is below that sentinel. *Reset Dose Syringe* clears the dose-volume threshold; it does not cancel an active stroke or immediately retract the syringe. Check subsequent physical movement and telemetry because buttons provide no execution acknowledgement.

The firmware parses `dose_time`, but the supplied dashboard has no schedule editor. Its matching-minute condition can repeat cycles without an execution latch; leave scheduling disabled until this behavior is corrected and tested. Supervise every bench cycle, record sensor-invalid/network-loss cases, and remove actuator power for a physical stop. Dashboard zeros, software angles, and status labels do not prove physiological validity or delivered volume.

APPENDIX B
PROJECT ARTIFACT TRACEABILITY

The report uses the following supplied artifact groups; generated build files are not experimental evidence:

- `Project_slides/`: the project presentation establishes its title, team, and concept; slide-only features require implementation evidence.
- `Project_images/1.jpeg` through `6.jpeg`: assembly, mechanisms, sensors, and electronics photographs.
- `Website_images/web_1.jpeg` through `web_7.jpeg`: dashboard, registration, and database captures; two pairs repeat the same screens.
- `medismart/`: sensor and actuator wiring PNGs, `firebase.json`, `public/` browser sources, and `medismartVscode/` firmware/build configuration.
- `CSE_4326_Project_Paper/`: the IEEE master, chapter files, metadata placeholders, `IEEEtran.cls`, `IEEEtran_HOWTO.pdf`, and `REPORT_INSTRUCTIONS.md`; `materials/` preserves the original template and source-review notes.

Replace the project repository and demonstration-video placeholders before submission [11], [12]. Keep original artifacts and distinguish photographed assembly, screenshot observations, source-derived behavior, and future physical measurements throughout the final report.